

CENTRO UNIVERSITÁRIO UNIFECAP

ENTREGA CONTÍNUA DE UMA PLATAFORMA DE PEDIDOS EM MICROSSERVIÇOS

Documentação técnica do projeto pedidos-veloz-platform

Relatório Técnico — Parte Prática
Cloud DevOps Engineering

São Paulo
2026

1 VISÃO GERAL DA SOLUÇÃO

Este relatório documenta as decisões técnicas do projeto entregue no repositório “pedidos-veloz-platform”, cobrindo os quatro microsserviços do desafio (Gateway, Pedidos, Pagamentos e Estoque), o banco PostgreSQL e um barramento de eventos para desacoplar Pedidos dos dois consumidores. O objetivo prático foi entregar um MVP funcional em quatro semanas que reduza risco de deploy, permita escalar sob demanda na campanha promocional e torne falhas rastreáveis entre serviços — sem adicionar complexidade que a equipe da Loja Veloz não consiga operar sozinha depois da entrega.

2 AMBIENTE LOCAL COM DOCKER COMPOSE

O arquivo `docker-compose.yml` sobe seis serviços — Postgres, o barramento de eventos NATS (com JetStream habilitado para persistir eventos), e os quatro microsserviços — em uma rede bridge isolada por projeto, com um único comando: `docker compose up -d --build`. As credenciais do banco ficam em um arquivo `.env`, nunca versionado (apenas um `.env.example` serve de modelo), e o schema do Postgres é aplicado automaticamente na primeira subida via volume montado em `/docker-entrypoint-initdb.d`. O healthcheck do Postgres é usado como condição de dependência: o serviço de Pedidos só inicia depois que o banco responde, evitando erro de conexão na primeira tentativa.

3 CONTEINERIZAÇÃO E VERSIONAMENTO

Todos os quatro serviços usam o mesmo padrão de Dockerfile multi-stage: um estágio “build” resolve as dependências Node.js e um estágio “runtime”, baseado em `node:20-alpine`, copia apenas o necessário para rodar — sem ferramentas de build, sem cache do npm. Três práticas de segurança foram aplicadas de forma consistente: usuário não-root criado explicitamente no Dockerfile (nenhum serviço roda como root), imagem base alpine para reduzir superfície de ataque e tamanho, e HEALTHCHECK nativo apontando para o endpoint `/healthz` de cada serviço. O versionamento de imagem segue o SHA do commit (não a tag “latest”) — isso é aplicado no pipeline de CI/CD, garantindo que o deploy em produção sempre referencia uma imagem rastreável até o código exato que a gerou.

4 KUBERNETES — PRODUÇÃO MÍNIMA

Os manifests em `k8s/base/` seguem a separação recomendada pela documentação oficial: ConfigMap para endereços internos e configuração não sensível, Secret para a string de conexão do banco (com observação explícita no próprio manifest de que valores reais

devem vir de um cofre externo no pipeline, nunca commitados em texto claro), Deployment com readiness e liveness probes apontando para /readyz e /healthz de cada serviço, e Service ClusterIP para comunicação interna. O Postgres foge do padrão dos demais: por precisar de identidade estável e armazenamento persistente por réplica, é declarado como StatefulSet com um volumeClaimTemplate, e não como Deployment comum.

Em segurança, o namespace inteiro (“pedidos-veloz”) foi rotulado com Pod Security Admission no perfil restricted, o que bloqueia automaticamente qualquer Pod que tente rodar como root, monte hostPath ou escale privilégios — reforçado individualmente em cada container com allowPrivilegeEscalation: false, readOnlyRootFilesystem: true e drop de todas as capabilities do Linux. A entrada externa é feita por um IngressRoute do Traefik, escolhido no lugar do ingress-nginx por já expor métricas e circuit breaker sem sidecar adicional, o que ajuda a manter o MVP simples de operar em quatro semanas.

5 CI/CD

O pipeline (.github/workflows/ci-cd.yml) roda em três estágios. O primeiro, em matriz para os quatro serviços, executa instalação de dependências, lint, testes e um scan de vulnerabilidade com Trivy que falha o build em achados CRITICAL ou HIGH — nenhuma imagem com vulnerabilidade grave conhecida chega ao próximo estágio. O segundo publica a imagem no registry e a assina digitalmente com cosign (Sigstore); essa assinatura é o principal diferencial de segurança do pipeline, porque o terceiro estágio, antes de aplicar qualquer manifest em staging, roda cosign verify e recusa a implantação se a imagem não tiver sido assinada por este mesmo pipeline — fechando a porta para alguém publicar manualmente uma imagem direto no registry e pular a esteira de validação. Segredos de registry e chave de assinatura ficam nos GitHub Secrets do repositório, nunca no código.

6 OBSERVABILIDADE, DEPLOY E ESCALA

Métricas, logs e traces dos quatro serviços são coletados via OpenTelemetry Collector (observability/otel-collector-config.yaml) e roteados para um stack Grafana LGTM (Loki, Grafana, Tempo, Prometheus): um painel único correlaciona os três sinais, o que ajuda uma equipe pequena a não precisar alternar entre três ferramentas diferentes no meio de um incidente. Cada serviço expõe /metrics no formato Prometheus e é anotado com prometheus.io/scrape: "true" para descoberta automática pelo Collector.

A estratégia de deploy é diferenciada por criticidade: o serviço de Pedidos usa deploy canário via Argo Rollouts (k8s/rollout/pedidos-rollout.yaml), liberando tráfego gradualmente — 10%, 30%, 60%, 100% — com pausas entre etapas e uma verificação

automática de taxa de erro via AnalysisTemplate consultando o Prometheus; se a taxa de erro 5xx do canário passar de 2%, o rollout é abortado antes de afetar todos os usuários. Gateway, Pagamentos e Estoque usam rolling update padrão do Kubernetes, considerado suficiente para o risco que cada um representa. Escala automática (HPA) foi configurada para Pedidos (2 a 10 réplicas) e Estoque (2 a 12 réplicas), os dois serviços com maior tendência de pico durante a campanha, com meta de 60-65% de utilização de CPU; VPA fica registrado como próximo passo recomendado, a ser ativado somente após um ciclo de observação com dados reais de uso em produção, para não gerar ajustes de recursos com base em uma amostra pequena demais.

7 INFRAESTRUTURA COMO CÓDIGO

O diretório terraform/ contém um esqueleto com módulo próprio “cluster”, reaproveitado entre staging e produção via variáveis (ambiente, região, contagem mínima e máxima de nós), com backend remoto em vez de state local — decisão que evita divergência entre ambientes e mantém o state fora do controle de versão. O provedor cloud definitivo (GKE, EKS ou AKS) ainda depende de uma decisão comercial da Loja Veloz, por isso o módulo documenta a forma esperada do node pool (autoscaling, subnets privadas, service account própria) em vez de fixar um provedor específico neste momento do projeto.

REFERÊNCIAS

GOOGLE CLOUD PLATFORM. microservices-demo (Online Boutique). Repositório GitHub, 2024. Disponível em: <https://github.com/GoogleCloudPlatform/microservices-demo>. Acesso em: 23 jul. 2026.

KUBERNETES. Documentação oficial. Disponível em: <https://kubernetes.io/docs/home/>. Acesso em: 23 jul. 2026.

DOCKER. Docker Compose documentation. Disponível em: <https://docs.docker.com/compose/>. Acesso em: 23 jul. 2026.

ARGO PROJECT. Argo Rollouts documentation. Disponível em: <https://argoproj.github.io/rollouts/>. Acesso em: 23 jul. 2026.

SIGSTORE. cosign documentation. Disponível em: <https://docs.sigstore.dev/cosign/overview/>. Acesso em: 23 jul. 2026.

TERRAFORM. Documentação oficial. Disponível em: <https://developer.hashicorp.com/terraform/docs>. Acesso em: 23 jul. 2026.

OPENTELEMETRY. Documentação oficial. Disponível em: <https://opentelemetry.io/docs/>. Acesso em: 23 jul. 2026.